Optimizing McDonald's Delivery Routes in Vancouver: A Heuristic Approach

Jianyi Chen, Jincheng Hong, Yicheng Huang, Tianyu Su

March 2025

Introduction

As one of the largest fast-food chains, McDonald's depends on an efficient supply chain to deliver fresh ingredients to its stores daily. A key challenge in this process is determining optimal delivery routes that minimize costs and travel time. In this project, we focus on the problem of planning efficient delivery routes for multiple McDonald's locations in Vancouver, supplied from a central depot or multiple depots.

In this project, we develop and analyze three models, each designed for different logistical scenarios, such as multiple delivery vehicles and multiple depots. Our approach applies principles from combinatorial optimization, incorporating routing strategies and greedy algorithms to construct efficient solutions.

1 Problem Statement

The goal of this project is to find optimal delivery routes for McDonald's ingredient distribution in Vancouver under different logistical constraints. Depending on the scenario, deliveries may be carried out by one or two vehicles, operating from either a single depot or dual depots. The objective is to minimize the total travel distance while ensuring that the routes are balanced, with each vehicle covering a similar distance. Given the complexity of the problem, we employ heuristic methods to obtain near-optimal solutions efficiently.

2 Methods

2.1 Problem Initialization

To construct a realistic road network, we use the `osmnx` package to retrieve and analyze street network data from OpenStreetMap. We begin by recording the latitude and longitude coordinates of each McDonald's store in downtown Vancouver, Vancouver, and East Vancouver using Google Maps. In total, we consider 24 McDonald's locations. For this model, we assume that the central depot is located at Vancouver International Airport (YVR).

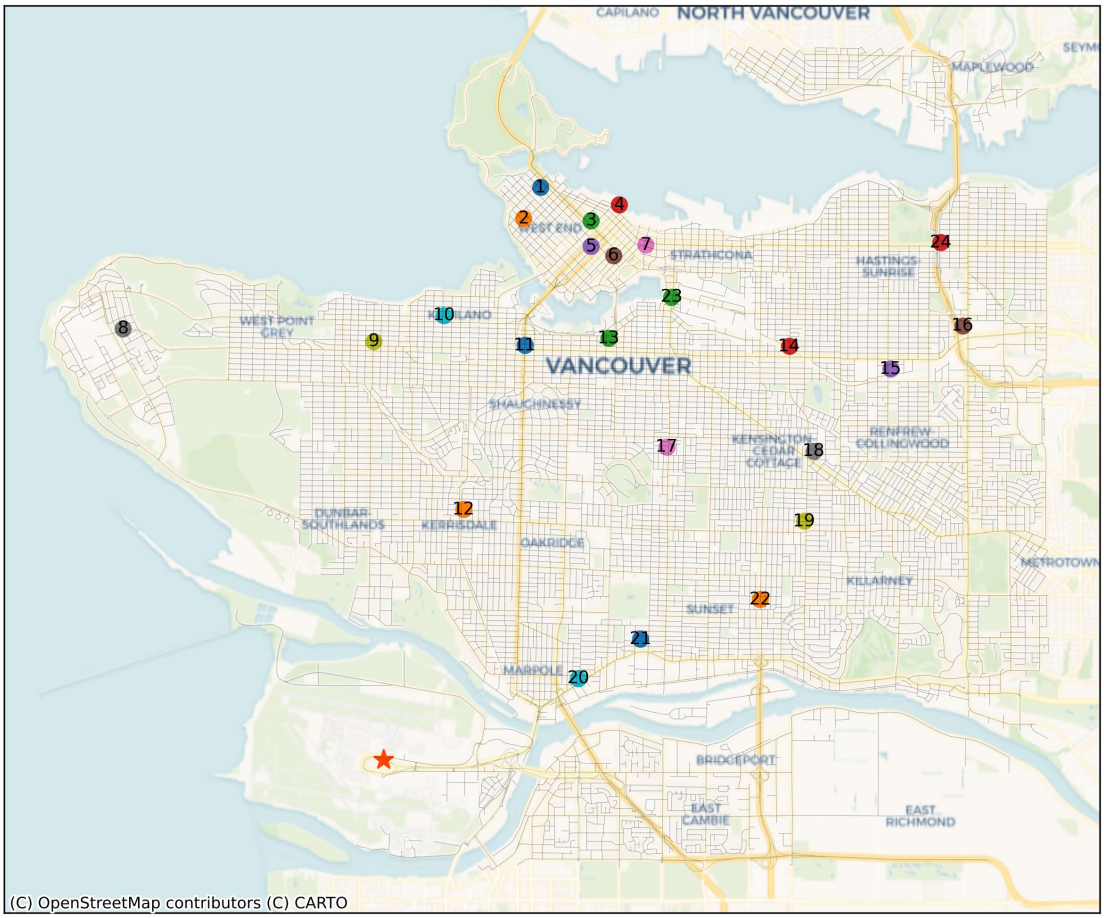


Figure 1: Map of Vancouver with McDonald's locations labeled

2.2 Assumptions and Constraints

- The delivery vehicle has sufficient capacity and fuel to complete all deliveries without restocking or refueling.
- Each store must be visited exactly once.
- The vehicle starts and ends at the central depot YVR.
- Travel occurs along the existing car roads.
- The shortest path between locations is determined based on road network distances by OpenStreetMap rather than Euclidean distance.
- Traffic conditions, weather, and other external disruptions are not considered.

2.3 Variables and Parameters

We define V as the set of all vertices v_i, and V represents the locations of all stores plus the central depot. The travel cost between any two locations is given by the cost matrix C, an $(n \times n)$ square matrix where each entry c_{ij} represents the travel cost or distance from vertex v_i to vertex v_j.

A tour T is defined as an ordered sequence of vertices that represents a complete delivery route, starting and ending at the depot. Mathematically, a tour can be written as $T = \{v_0, v_1, v_2, \ldots, v_n, v_0\}$, where v_0 denotes the depot and the remaining vertices $v_1, v_2, \ldots, v_n$ correspond to the stores.

Symbol	**Description**
V	Set of all vertices (stores and depot), $V = \{v_0, v_1, v_2, \ldots, v_n\}$
C	Cost matrix $(n \times n)$, where c_{ij} is the travel cost (distance) from v_i to v_j
T	Tour, a sequence of vertices: $\{v_0, v_1, v_2, \ldots, v_n, v_0\}$

Table 1: Variables

The objective of this model is to minimize the total travel cost, where the vehicle departs from the depot, visits each store exactly once, and returns to the depot. Formally, if c_{ij} represents the travel cost from vertex v_i to vertex v_j, the total cost of a given tour T can be expressed as:

$$\text{Cost}(T) \;=\; \left(\sum_{i=0}^{n-1} C[v_i, v_{i+1}]\right) + C[v_n, v_0]$$

Note that the last term represents the final return to the depot.

3 Basic Model: Single-Vehicle Routing with a Central Depot

In this scenario, we construct a fundamental model where a single vehicle departs from a central depot, visits each store exactly once, and returns to the depot. This approach is based on concepts covered in *Unit 2, Lecture 3*.

3.1 Designing idea

To find the shortest route, we simply apply **the nearest-neighbor method**:

1. Start at the depot (initial vertex v_0).
2. At each step, choose the closest unvisited store (for a specific v_i, find the vertex v_j minimizing $C[v_i, v_j]$ among all unvisited vertices).
3. Add v_j to the tour, mark that store as visited and move there.
4. Repeat until all stores have been visited.
5. Return to the depot to complete the tour.

Through this heuristic algorithm, we should be able to find a complete and decent delivery tour.

3.2 Results

This model successfully finds a route:

Tour	[0, 20, 21, 22, 19, 18, 14, 15, 16, 24, 7, 6, 5, 3, 4, 1, 2, 11, 13, 23, 17, 12, 9, 10, 8, 0]
Cost(km)	75.18

Table 2: Tour for Single Vehicle Routing

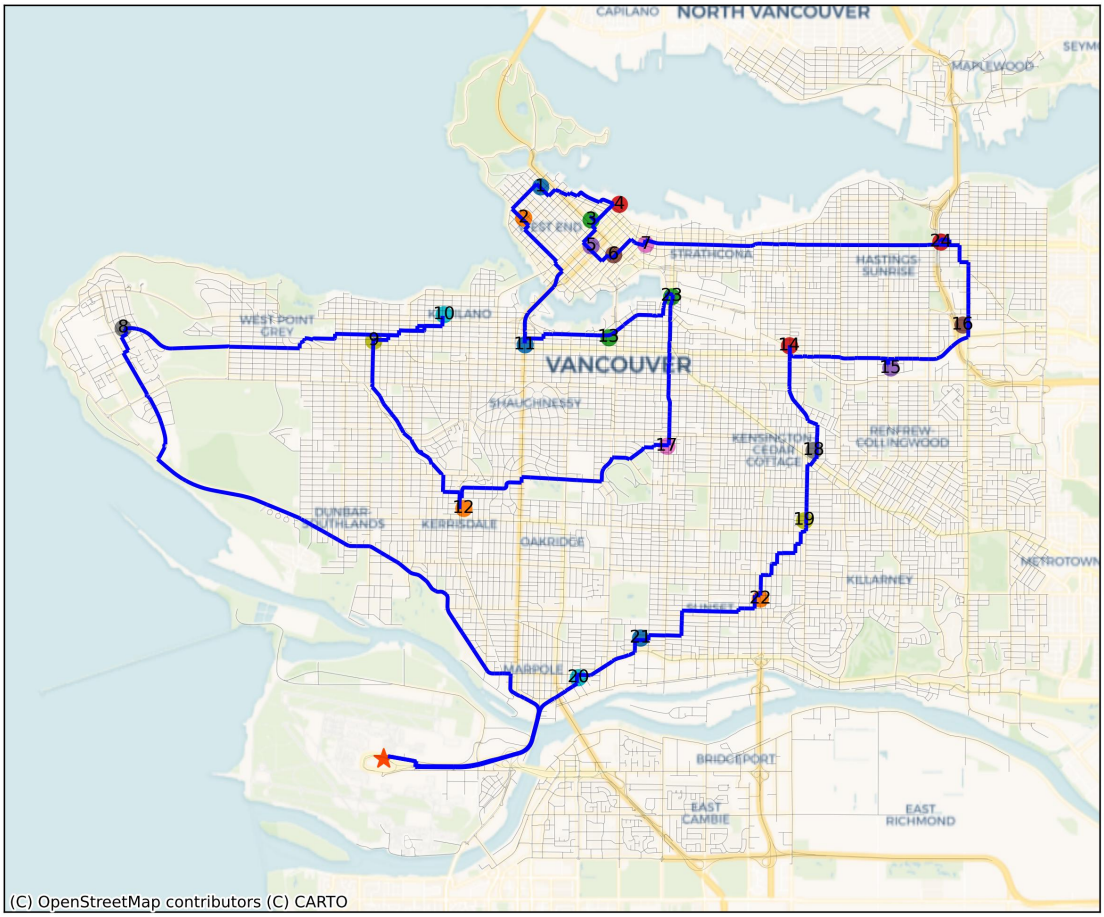


Figure 2: Tour for Single Vehicle Routing

3.3 Analysis and Evaluation

The model successfully finds a nearly optimal route and is easy to understand and interpret: it simply constructs the path by iteratively selecting the nearest unvisited location. However, an issue is that the model strict adheres to the nearest neighbor heuristic. For example, the model chose 12-9-10-8 since store 9 is closer to store 12 than store 10. But this choice results in a small overlap in the route when traveling from store 9 to store 10 and then back to location 8. A shorter path would be 12-10-9-8, which avoids overlapping of routes.

While this model performs well in this simplified scenario, it has limitations. One key limitation is the lack of a randomized starting point for the nearest neighbor algorithm. Since the nearest neighbor algorithm is highly sensitive to the starting location, always beginning from the depot may lead to suboptimal solutions. A possible improvement is to first construct a tour among the stores before integrating the depot. That is, instead of always starting at the depot, we could first identify an optimal route between the stores and then break the edge between stores 20 and 12 and connect them to YVR. Despite this limitation, the algorithm still provides a nearly optimal solution given the specific map and store locations used.

In real-world applications, delivery operations rarely involve just a single vehicle. Additionally, constraints such as restricted delivery time windows often requires multiple vehicles operating simultaneously. To address these more complex scenarios, we extend our model to incorporate two vehicles departing from the same depot.

4 Intermediate Model: Dual-Vehicle Routing with a Shared Depot

In this scenario, we have two vehicles with the same depot. They simultaneously visit all stores and then return to the depot. The goal is to optimize their routes to minimize total travel distance while balancing the distance of the two routes.

To formalize this, we define two separate tours, T_1 and T_2, corresponding to the routes taken by each vehicle:

Symbol	Description
T_1	Tour for the first vehicle: $\{v_0, v_{a_1}, v_{a_2}, \ldots, v_{a_k}, v_0\}$
T_2	Tour for the second vehicle: $\{v_0, v_{b_1}, v_{b_2}, \ldots, v_{b_m}, v_0\}$

Table 3: Tour Definitions for Dual-Vehicle Routing

The objective is to minimize the total travel cost, which is the sum of the distances of both tours:

$$\text{Total Cost} = \text{Cost}(T_1) + \text{Cost}(T_2)$$

And we also want the distance of each route to be balanced,

$$\text{Cost}(T_1) \approx \text{Cost}(T_2)$$

To do this, we introduce two heuristic algorithms based on the nearest neighbor approach to find routes for both vehicles.

4.1 The Splitting Algorithm

4.1.1 Designing idea

The idea of the splitting algorithm is to evenly distribute the delivery workload between the two vehicles. Let n be the total number of stores, and let m_1 and m_2 denote the number of stores assigned to vehicles 1 and 2, respectively. By default, we set

$$m_1 = \lfloor n/2 \rfloor, \quad m_2 = n - m_1$$

The first vehicle visits m_1 stores determined by the nearest neighbor heuristic and then returns to the depot. The second vehicle then follows the same heuristic to visit the remaining stores.

We also incorporate a parameter a that allows slight adjustments to the store distribution,

$$m_1 = \lfloor n/2 \rfloor + a, \quad m_2 = n - m_1$$

This give us some flexibility.

Algorithm Steps

1. Initialize two tours, T_1 and T_2, both starting at the depot v_0.
2. Assign the first m_1 stores to vehicle 1 using the nearest neighbor method.
3. Assign the remaining m_2 stores to vehicle 2 using the same nearest neighbor heuristic.
4. Close both tours by returning each vehicle to the depot v_0.

4.1.2 Results

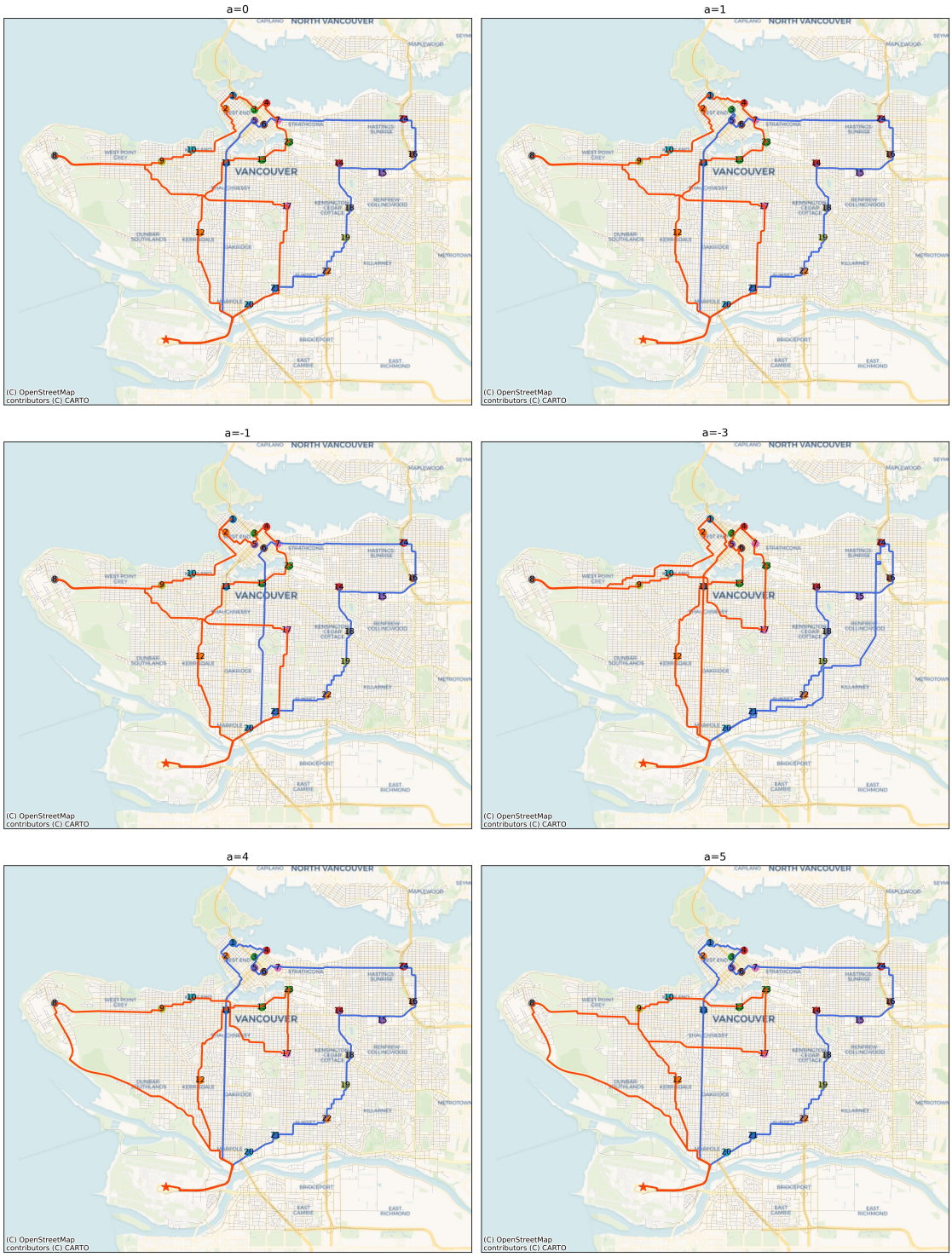


Figure 3: The Set of Routes For Different a Value

a **value**	**Tours**
$a = 0$	T_1: [0, 20, 21, 22, 19, 18, 14, 15, 16, 24, 7, 6, 5, 0]
	T_2: [0, 12, 11, 13, 23, 4, 3, 1, 2, 10, 9, 8, 17, 0]
$a = 1$	T_1: [0, 20, 21, 22, 19, 18, 14, 15, 16, 24, 7, 6, 5, 3, 0]
	T_2: [0, 12, 11, 13, 23, 4, 1, 2, 10, 9, 8, 17, 0]
$a = -1$	T_1: [0, 20, 21, 22, 19, 18, 14, 15, 16, 24, 7, 6, 0]
	T_2: [0, 12, 11, 13, 23, 4, 3, 5, 2, 1, 10, 9, 8, 17, 0]
$a = -3$	T_1: [0, 20, 21, 22, 19, 18, 14, 15, 16, 24, 0]
	T_2: [0, 12, 11, 13, 6, 5, 3, 4, 7, 23, 17, 10, 9, 8, 2, 1, 0]
$a = 4$	T_1: [0, 20, 21, 22, 19, 18, 14, 15, 16, 24, 7, 6, 5, 3, 4, 1, 2, 0]
	T_2: [0, 12, 11, 13, 23, 17, 10, 9, 8, 0]
$a = 5$	T_1: [0, 20, 21, 22, 19, 18, 14, 15, 16, 24, 7, 6, 5, 3, 4, 1, 2, 11, 0]
	T_2: [0, 12, 9, 10, 13, 23, 17, 8, 0]

Table 4: Tours for Different a Values

a **value**	**Number of Stores Visited**	**Costs (km)**	**Total Cost (km)**
0	(12, 12)	(41.89, 52.85)	94.73
1	(13, 11)	(43.46, 52.6)	96.06
-1	(11, 13)	(41.33, 54.65)	95.98
-3	(9, 15)	(40.05, 61.52)	101.58
4	(16, 8)	(46.61, 46.72)	93.32
5	(17, 7)	(46.61, 48.63)	95.23

Table 5: Comparison of Different a Values

4.1.3 Analysis and Evaluation

The splitting algorithm successfully generates routes for both vehicles, ensuring that all stores are covered. While the routes are generally reasonable, some unnecessary overlaps occur. For example, in the $a = 4$ model, the orange vehicle could follow the path $12 \to 17 \to 23 \to 13 \to 11 \to 10$, which would avoid the redundant travel.

Another limitation is the distribution of stores between the two vehicles. In the $a = 4$ model, for example, the blue vehicle passes store 10 but does not include it in its route because it has already reached its assigned quota of stores. As a result, both vehicles end up covering areas in both downtown and the east side of Vancouver, leading to repeated routes.

The choice of parameter a significantly affects the results. When a is positive, more stores are assigned to the blue vehicle; when negative, more are assigned to the orange vehicle. We observed that with $a = 0$, both vehicles must travel downtown, which is unnecessary. To improve this, we tested $a = -3$, where the blue vehicle stops before entering downtown, and $a = 4$, where it completes all downtown deliveries before returning to the depot. The results indicate that $a = -3$ does not yield an better solution, but $a = 4$ provides better results, achieving the lowest total cost among all tested values.

However, upon closer inspection of the $a = 4$ solution, we noticed that while the blue vehicle passes store 10, it does not visit it, forcing the orange vehicle to revisit the area. To address this, we increased a to 5, but the results did not improve (refer to the plot above). This suggests that the nearest neighbor heuristic, in its current form, is not intelligent enough.

Overall, by adjusting a, we found that $a = 4$ produces a reasonably efficient model. It would be even better with some manual adjustments to the routes. For future improvements, we can explore randomized starting point in the nearest neighbor algorithm, which we believe could help with store assignments and further optimize the solution.

4.2 The Alternating Algorithm

4.2.1 Designing idea

We use a simple greedy strategy in which the two vehicles alternate in claiming the next delivery location:

1. Initialize two empty tours T_1 and T_2, both starting at v_0.
2. Then, add the nearest unvisited store to the last location to T_1, and remove it from the unvisited set.

3. Next, add the nearest unvisited store to the last location to T_2, and remove it from the unvisited set.

4. Repeat the alternating selection until there are no unvisited stores left.

5. Close both tours by returning each vehicle to the depot v_0.

4.2.2 Results

We succeed in finding the shortest route of the two vehicles and we can find that these routes do not include unnecessary overlapping parts. Likewise, the blue line is the route of the first car and the red line is the route of the second car.

Tours	T_1: [0, 20, 22, 19, 18, 14, 15, 16, 24, 13, 11, 10, 12, 0] T_2: [0, 21, 17, 23, 7, 6, 5, 3, 4, 1, 2, 9, 8, 0]
Distances(km)	45.56, 45.61
Total Distances(km)	91.17

Table 6: Results for the Alternating Algorithm

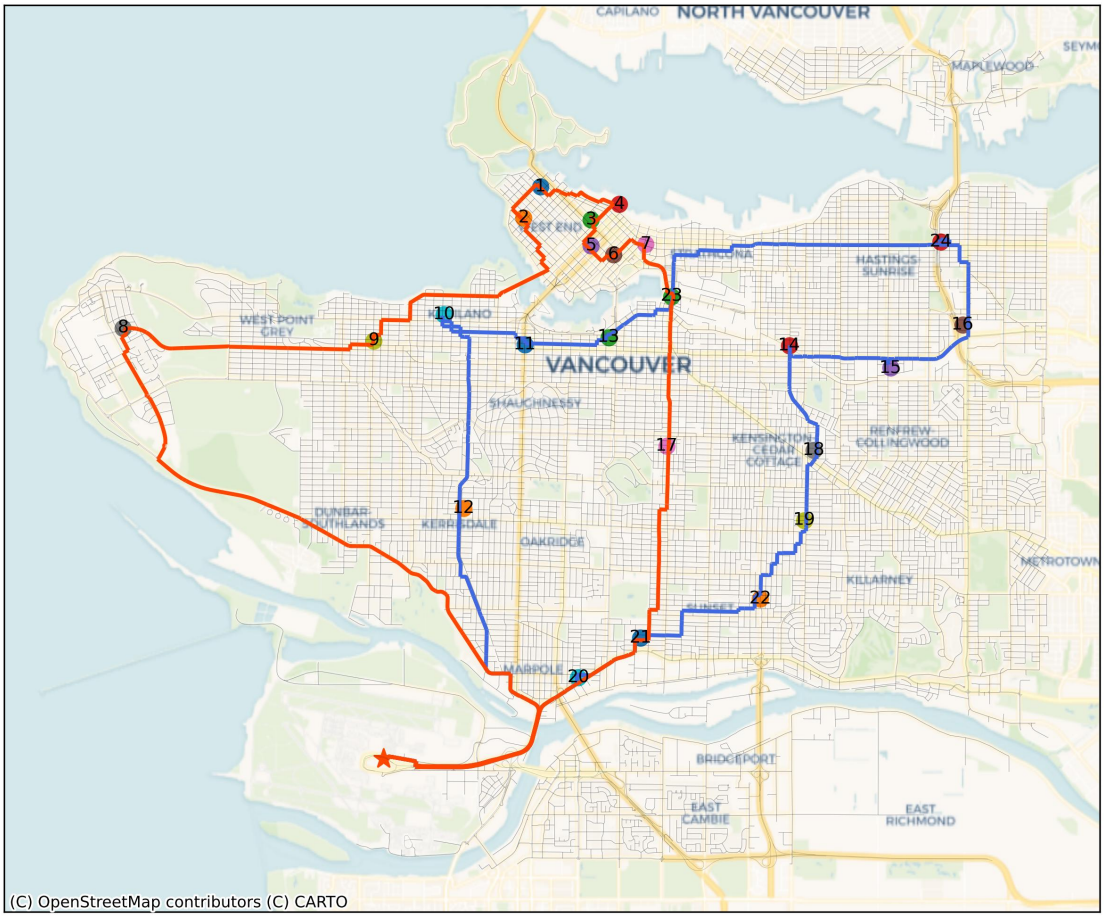
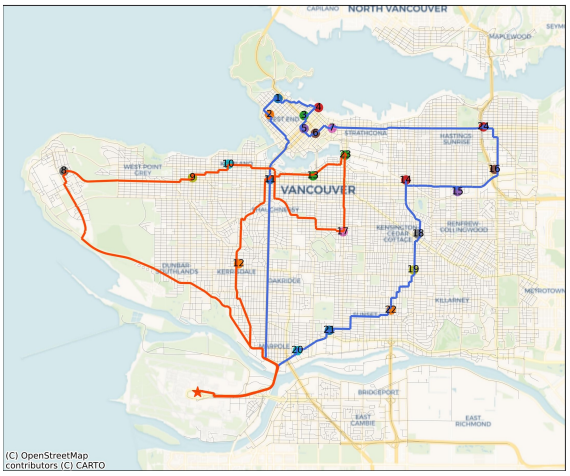


Figure 4: Routes for the Alternating Algorithm

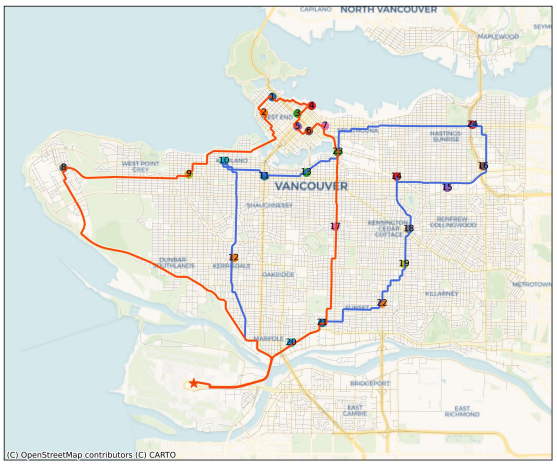
4.2.3 Analysis and Evaluation

The alternating algorithm successfully constructs two valid routes by allowing each vehicle to alternately select its next stop. Surprisingly, this approach performs exceptionally well. The resulting routes are well-balanced, with no obvious inefficiencies or unnecessary detours.

Notably, this algorithm achieves a lower total travel cost (91.17 km) compared to the splitting algorithm (93.32 km) while also ensuring that the distances covered by both vehicles remain fairly equal. This balance is a desirable outcome, as it optimizes overall efficiency and workload distribution. The alternating algorithm is highly effective given the specific layout of McDonald's stores in Vancouver.



(a) Splitting



(b) Alternating

Algorithm	**Number of Stores Visited**	**Costs (km)**	**Total Cost (km)**
Splitting	(12, 12)	(41.89, 52.85)	94.73
Splitting	(16, 8)	(46.61, 46.72)	93.32
Alternating	(12, 12)	(45.56, 45.61)	91.17

Table 7: Comparison of the Splitting and Alternating Algorithm

4.3 Comparison between two algorithms

From the data sheet, it shows that the total distance traveled by the alternating algorithm is 91.17 km, which is shorter than the 94.73 km of the splitting algorithm for the same task allocation (that is, 12 locations visited by each vehicle). This suggests that the alternating algorithm may be more effective in optimizing the path and can reduce the distance traveled by vehicles. However, when the splitting algorithm adjusts the location assignment strategy (16, 8), its total distance traveled decreases to 93.32 km, which is very close to the result of the alternating algorithm; suggesting that the performance of the splitting algorithm can be improved and come close to the optimization level of the alternating algorithm by appropriately adjusting the task assignment strategy.

1. Splitting method - This algorithm first performs a static splitting and then optimizes the path for each vehicle separately. The advantage of this method is that it is computationally efficient and generates solutions quickly, but usually only local optima can be found because the initial division affects the final result.

2. Alternating method - This algorithm adjusts alternately after the initial path planning,

making the assignment of tasks to the two vehicles dynamic and thus helping to reduce the overall distance traveled. This method can obtain a GLOBAL optimal solution in some cases.

5 Advanced Model: Dual-Vehicle Routing with Dual Depots

In this model, we change the problem to have two depots instead of one. Thus, a new challenge is to determine where these depots should be placed and how to assign stores to them efficiently before planning delivery routes.

5.1 Designing idea

Considering the division problem, we apply k-means clustering to divide all stores into two natural clusters. We then designate the centroids of the two clusters as the two depots. And once the depot locations are determined, each store is assigned to its nearest depot, and we solve a separate routing problem for each cluster.

1. Perform standard K-means, generating initial clusters and centers.
2. Check balance, we have set the size of difference between two clusters need under 2.
3. Determine which cluster is larger or smaller.
4. Mainly reassign points from large to small until the difference meets the difference constraint which repeats works here by computer calculating the loop. (follow step 2).
5. Update centers (depot location information) by information from the updated cluster.
6. Repeat above steps until satisfied.
7. When we determined cluster assignments, the problem could seem as solving two independent single-vehicle routing problems with a central depot. We would directly apply the standard routing model and algorithm.

5.2 Results

We successfully determined the shortest routes for the two vehicles, each departing from a different depot. The blue star marks the starting point of Route 1, while the red star marks

the starting point of Route 2. The vehicles serve 13 stores and 11 stores, respectively. (which is under our `max_difference` constraint)

Tours	T_1: [0, 11, 13, 6, 5, 3, 4, 7, 1, 2, 10, 9, 12, 8, 0] T_2: [0, 18, 19, 22, 21, 20, 17, 23, 14, 15, 16, 24, 0]
Distances(km)	38.07, 34.54
Total Distances(km)	72.61

Table 8: Results for the Alternating Algorithm

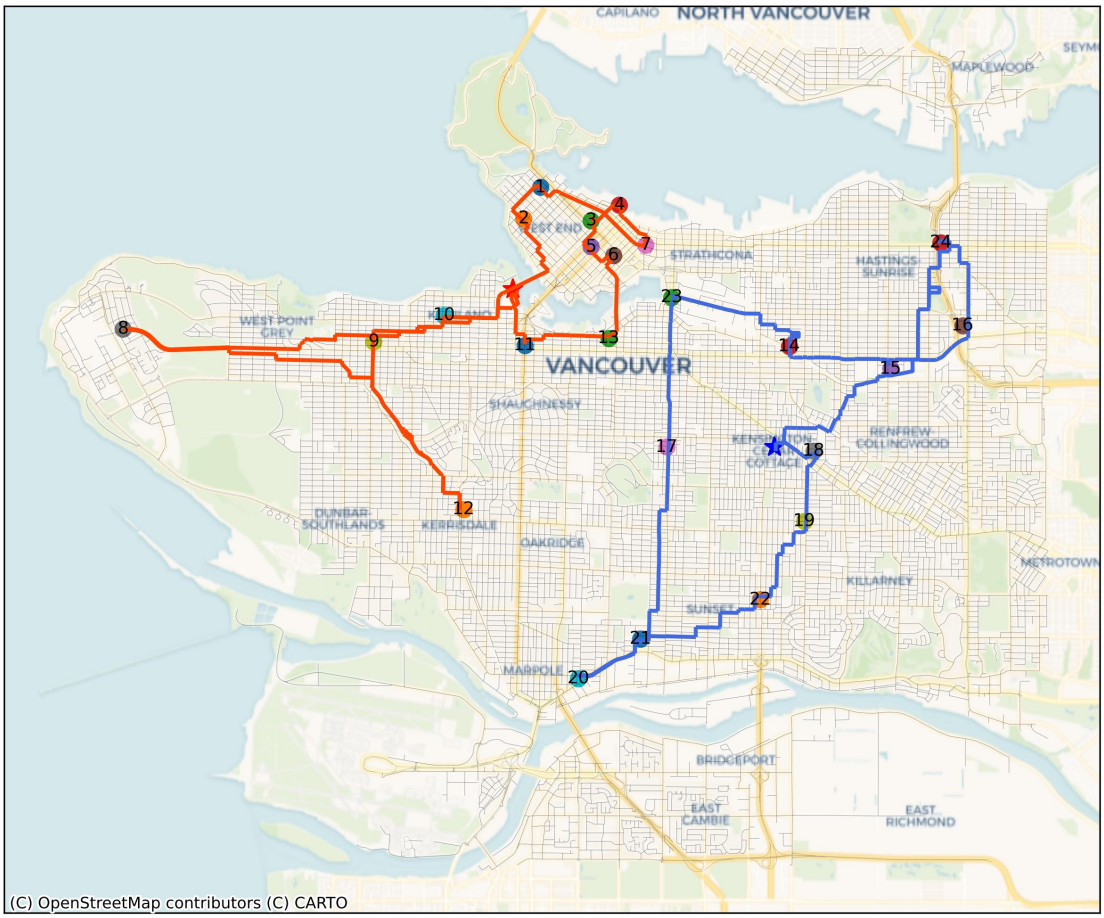


Figure 5: Optimized delivery routes for two depots after balanced K-Means clustering

5.3 Analysis and Evaluation

This model is very attractive, and the key strength of this approach is that it strategically determines the locations of the depot before planning the routes. Unlike the single-depot models where all deliveries originate from a fixed point, this model adapts the depots to the actual store distribution. Meanwhile, when the number of depots increases, it still helps us to find the optimal depot location and draw the optimal routes. By comparing with the previous model, we can clearly identify that this model returns much smaller distances than previous results, which is a good consequence.

However, this method faces challenges such as computational complexity and storage limitations. As the number of stores increases, the computational cost of K-means clustering and route optimization also rises. Therefore, it may be more suitable for training on small-sized datasets.

Conclusion

In this project, we explore combinatorial optimization methods to design efficient delivery routes for McDonald's locations across Vancouver. In each model, we address the routing problem with different degrees of complexity and employ heuristic algorithms adapted to each context.

In the first model of a single vehicle routing with a central depot, we use the nearest-neighbor method to find the shortest route. While simple and intuitive, this method is clearly suboptimal, leading to inefficiencies such as repeated routes due to its rigid greedy selection.

Then, we develop an intermediate model using two delivery vehicles from the same depot. And we have two algorithms to solve it. The Splitting Algorithm has similar problems as the basic model because it also applies a nearest-neighbor logic to each vehicle separately, which leads to overlaps and an uneven distribution. Conversely, the Alternating Algorithm significantly improves performance by allowing vehicles to alternately select the next closest unvisited store. This method effectively minimizes unnecessary route overlaps and produces a shorter combined total distance compared to the splitting method.

Finally, we introduce an advanced model with dual depots and apply K-means clustering to position the depots. This unsupervised learning method significantly optimizes the overall delivery layout. By clustering stores into two balanced partitions, we simplify the routing problem within each cluster. This approach substantially reduces the total travel distance to approximately 73 km. However, route optimization within each cluster still employs the nearest-neighbor heuristic, so it retains some inefficiencies and route repetitions observed

previously.

For future work, we recommend exploring more sophisticated routing algorithms to probably get better results. For example, we can add the number of vehicles to see how much better it is and when it is no longer beneficial to add vehicles. It would also be beneficial to incorporate real-world constraints such as traffic congestion, vehicle capacity limitations, and delivery time windows to further improve route optimization.

References

[1] GeeksforGeeks. (2025). *K-Means Clustering - Introduction*. GeeksforGeeks. https://www.geeksforgeeks.org/k-means-clustering-introduction/

[2] Walls, P. (2025). *OpenStreetMap and Travelling Salesperson*. Unpublished class notes, University of British Columbia.